

Bug ID	Title	Short description	Run command	Waveform
b1	Count output shows wrong value after main_load register is written	According to the section "Loading main load value", the counter value should update directly when a new value is written to the counter_main_load_config_reg register. However, when 0xC2 is written, the output shows 0xC1, one less than the expected value, even before the counter is enabled.	make run TEST=mode1_free_run_up_test	verif/bug_waves/bug1.png
b2	Counter does not increment in free run UP mode after enable	The section "Enabling and disabling the counter block" states that once the counter is enabled, it should count up or down based on the configured direction and mode. In the observed case, enable is asserted at 415 ns, the counter is set to free run UP mode, but the output remains stuck at 0xC1 until the end of simulation at 2245 ns. Additionally, the section "Changing the Number of cycles for the counter block" explains that the counter_number_of_cycles_config_reg register determines how many rising clock edges are required before the next count update. With num_cycles = 5, an increment should occur every 6 rising edges. From enable to the end of simulation, 180 clock cycles occur, enough for about 30 increments, yet no increments are observed.	make run TEST=mode1_free_run_up_test	verif/bug_waves/bug2.png
b3	cycles_count loaded with max value instead of zero on first enable	Counter configured in free run UP mode (mode=2'b00, dir=1), main_load=0xC2 (194), number_of_cycles_val=0x05. Enable asserts at 405ns. Per spec, once enabled the cycle counter should start from zero and count up one step per rising clock edge until it hits the threshold of 0x05, then update the count output. This should keep repeating to produce steady increments. Instead, the waveform shows cycles_count jumping straight from 0x00 to 0x05 the moment enable is asserted at 405ns, loading the threshold value directly into the cycle counter instead of starting from zero. This makes the counter update on the very first clock after enable rather than after 5 edges. After that the cycle counter never works correctly again and the count output freezes. Log confirms first COUNT MISMATCH at 425ns: expected=194 got=195, then expected=194...223 got=193 repeating until end of simulation at 2245ns with no further increments observed.	make run TEST=mode1_free_run_up_test	verif/bug_waves/bug3.png

b4	Counter output is one step ahead of the expected value on every update	According to the spec section "Enabling and disabling the counter block", once enabled the counter should increment by exactly 1 step per configured number of clock edges. Counter configured in free run UP mode, number_of_cycles=5, enable asserts at 405ns. The waveform shows that every time the counter is due to update its value, it outputs a value that is 1 higher than expected for one clock cycle, then on the next clock it settles 1 below the expected value. For example at 475ns, when the expected value is 195, the counter first outputs 196, then on the next clock outputs 194. The counter never lands on the correct value, it always overshoots by 1 then undercorrects by 1, repeating this pattern continuously to the end of simulation.	make run TEST=mode1_free_run_up_test	verif/bug_waves/bug4.png
b5	cycles_count counts downward instead of upward, causing the counter output to freeze	According to the spec section "Changing the Number of cycles for the counter block", cycles_count should increment from 0 up to the configured threshold before updating the counter output. However, the waveform shows that immediately after enable asserts at ~420ns, cycles_count begins decrementing every clock: FF→FE→FD→FC→FB→FA instead of counting upward. Since it counts in the wrong direction it never reaches the threshold of 0x05, causing the count output to remain completely frozen at 0xC2 from enable assertion all the way to the end of simulation.	make run TEST=mode1_free_run_up_test	verif/bug_waves/bug5.png
b6	Counter counts down instead of up when direction is set to UP	According to the spec section "Change the count direction", writing 1 to bit [2] of the counter_mode_control_config_reg register configures the counter to count UP. The counter was configured with dir=1 (UP), mode=00 (free run), main_load=0xC2 (194), number_of_cycles=5. With this configuration, once enabled, next_count should increment by 1 every time cycles_count completes a full cycle from 0 up to the threshold of 0x05. The waveform shows cycles_count correctly counting upward 01→02→03→04→05→00 and repeating, confirming the cycle threshold mechanism is working. However, next_count is observed moving C2→C1→C0→BF, decrementing by 1 on each cycle completion instead of incrementing. count follows the same downward progression one step behind: C2→C1→C0→BF. The counter is clearly running in the wrong direction — counting DOWN despite being configured for UP	make run TEST=mode1_free_run_up_test	verif/bug_waves/bug6.png

b7	Counter does not wrap to main load value in special single wrap point mode	According to the spec section "4-special single wrap point mode", when the counter reaches MAX_VALUE (0xFF) or 0, it must wrap back to the configured main_load value. The counter was configured with mode=11 (special single wrap point), dir=1 (UP), main_load_val=0xFD (253), number_of_cycles=5. With this configuration, the expected counting sequence is 253→254→255→253→254→255, wrapping back to 0xFD every time 0xFF is reached. The waveform shows cycles_count correctly cycling through its threshold, and main_load_val is stable at 0xFD throughout. However, when next_count reaches 0xFF, instead of wrapping to 0xFD, it rolls over to 0x00 and continues incrementing freely: FE→FF→00→01→02→03... The wrap condition at MAX_VALUE is completely ignored and the counter never returns to the main load value.	<pre>make run TEST=mode4_special_wrap_up_test</pre>	verif/bug_waves/bug7.png
b8	Counter increments by 2 instead of 1 in single wrap point mode	According to the spec section "Enabling and disabling the counter block", once enabled the counter should increment by exactly 1 step per configured number of clock edges. The counter was configured with mode=01 (single wrap point), dir=1 (UP), main_load_val=0x64 (100), number_of_cycles=5. With this configuration, next_count should increment by 1 every time cycles_count completes a full cycle from 0 up to the threshold of 0x05. The waveform shows cycles_count correctly counting 01→02→03→04→05→00 and repeating, confirming the cycle threshold mechanism is working. However, next_count and count are both observed incrementing by 2 on every cycle completion: 0x64→0x66→0x68→0x6A, skipping every odd value entirely.	<pre>make run TEST=mode2_single_wrap_up_test</pre>	verif/bug_waves/bug8.png
b9	Counter fails to wrap at main load boundary in double wrap point mode	According to the spec section "3-Double wrap point mode", when the counter is counting UP in the inner region and reaches the main load value, it must wrap back to the secondary load value. The counter was configured with mode=10 (double wrap point), dir=1 (UP), main_load=0xDC (220), secondary_load=0x64 (100), number_of_cycles=5. The inner region is between secondary_load (0x64) and main_load (0xDC), so when counting UP the counter should wrap from 0xDC back to 0x64. The waveform shows cycles_count correctly cycling through its threshold and the counter incrementing correctly by 1 through the inner region. However, at ~7743ns when next_count reaches 0xDC (main_load), instead of wrapping to 0x64 (secondary_load), count transitions DC→DD and next_count continues to 0xDD, completely ignoring the wrap condition and counting past the main load boundary. The log confirms the first mismatch at 7755ns: expected=100 (0x64, the correct wrap target) but got=221 (0xDD, one step past main_load), with both values then incrementing in parallel by 1 for the rest of simulation.	<pre>make run TEST=mode3_double_wrap_inner_up_test</pre>	verif/bug_waves/bug9.png

b10	Counter counts past secondary load value instead of wrapping to main load in double wrap mode	According to the spec section "3-Double wrap point mode", when the counter is counting UP in the inner region and reaches the secondary load value, it must wrap back to the main load value. The counter was configured with mode=10 (double wrap), dir=UP, main_load=0x0A (10), secondary_load=0x14 (20), number_of_cycles=5. The waveform shows cycles_count correctly cycling through its threshold. However, at ~1140ns when count reaches 0x14 (secondary_load), instead of wrapping back to 0x0A (main_load), count steps forward to 0x15 at ~1144ns, completely ignoring the wrap boundary.	make run TEST=mode3_double_wrap_dir_c hange_test	verif/bug_waves/bug10.png
b11	Counter wraps to 0xFF instead of secondary load when counting DOWN in double wrap mode	According to the spec section "3-Double wrap point mode", the inner region is defined between main_load and secondary_load values. When the counter is counting DOWN inside the inner region and reaches the main load value, it must wrap back to the secondary load value and continue counting down from there. The counter was configured with mode=10 (double wrap), dir=DOWN, main_load=0x0A (10), secondary_load=0x14 (20), number_of_cycles=5. So the expected behavior is that the counter counts down inside the inner region 0x14→0x13→...→0x0A, then wraps back to 0x14 and repeats. However, the waveform shows the counter completely ignoring the main load boundary at 0x0A and continuing to count down freely: 0x0A→0x09→...→0x00, and then at ~3300ns instead of wrapping to 0x14, the counter wraps to 0xFF, as if it were in free running mode with no wrap boundaries at all.	make run TEST=mode3_double_wrap_dir_c hange_test	verif/bug_waves/bug11.png
b12	Counter immediately wraps from main load to secondary load instead of counting through the inner region in double wrap mode	According to the spec section "3-Double wrap point mode", when the counter starts at the main load value and is counting UP in the inner region, it should increment normally through the inner region toward the secondary load value. The counter was configured with mode=10 (double wrap), dir=UP, main_load=0x64 (100), secondary_load=0xDC (220), number_of_cycles=5. The waveform shows that at ~535ns, when cycles_count completes its threshold cycle, instead of incrementing from 0x64 to 0x65, next_count immediately jumps to 0xDC (secondary_load), skipping the entire inner region.	make run TEST=mode3_double_wrap_inner _up_test	verif/bug_waves/bug12.png

b13	Counter oscillates between main load and MAX value instead of counting through the upper region in single wrap point mode	According to the spec section "2-Single wrap point mode", when the counter is counting UP and reaches the main load value, it must wrap to the maximum value (0xFF) and then continue counting upward from there. The counter was configured with mode=01 (single wrap point), dir=1 (UP), main_load_val=0xE0 (224), number_of_cycles=1. The expected behavior is that once the counter wraps to 0xFF it continues incrementing normally. However, the waveform shows that at ~2378ns, when count reaches 0xE0 (main_load_val), it wraps to 0xFF, but then on the very next wrap cycle it immediately returns to 0xE0 again, creating a permanent oscillation: 0xE0→0xFF→0xE0→0xFF repeating forever and never making forward progress through the upper region.	make run TEST=mode2_single_wrap_at_m ax_up_test	verif/bug_waves/bug13.png
b14	Counter wraps to 0x00 instead of MAX value when reaching main load counting UP in single wrap point mode	According to the spec section "2-Single wrap point mode", when the counter is counting UP and reaches the main load value, it must wrap to the maximum value (0xFF) and continue counting upward through the upper region (main load – 2^{DATA_WIDTH}-1). The counter was configured with mode=01 (single wrap point), dir=1 (UP), main_load_val=0xE0 (224), number_of_cycles=1. The expected behavior is that once count reaches 0xE0 it wraps to 0xFF and continues incrementing from there. However, the waveform shows that at ~2378ns, when count reaches 0xE0 (main_load_val), it wraps to 0x00 instead of 0xFF, then continues incrementing freely: 0x00→0x01→0x02→0x03, as if it had switched to the lower region.	make run TEST=mode2_single_wrap_at_m ax_up_test	verif/bug_waves/bug14.png
b15	AGAIN, Counter oscillates between MAX value and main load instead of counting through the upper region in single wrap point mode	According to the spec section "2-Single wrap point mode", when the counter is counting UP in the upper region and reaches the maximum value (0xFF), it should continue incrementing normally from there. The counter was configured with mode=01 (single wrap point), dir=1 (UP), main_load_val=0xE0 (224), number_of_cycles=1. This same oscillation symptom was previously observed and documented as bug 13. After fixing bug 13, bug 14 appeared and was fixed, but upon re-running the test the same oscillation between 0xE0 and 0xFF returned. Since the symptom was identical to bug 13, the waveform was re-examined more carefully alongside the spec to understand what else could cause the wrap protection to fail. Observing the prev_main_load_wrap signal in the waveform at 2350ns–2560ns, it can be seen that it pulses high for exactly one clock each time the counter arrives at 0xFF, then immediately drops back to 0 on that same clock, so on the very next cycle the counter treats 0xFF as a fresh wrap point and snaps back to 0xE0.	make run TEST=mode2_single_wrap_at_m ax_up_test	verif/bug_waves/bug15.png